

Hybrid Quantum 4-Sieve

MohamadAli Khajeian

School of Engineering Sciences, College of Engineering, University of Tehran

khajeian@ut.ac.ir

September 29, 2026

Abstract

We present an adaptation of the quantum random walk (QRW) 2-sieve to serve as the inner two-list subroutine inside a quantum 4-sieve. Inspired by the classical 4-sieve, we construct the quantum 4-sieve and use the QRW 2-sieve as its inner two-list subroutine. The adaptation preserves the Johnson graph, the second-level random product code, and the quantum data structure of the original algorithm, and modifies only the marked condition and the probability that a random cross-pair is a solution. We provide the complete algorithms, discuss the modifications required, and give a full complexity analysis for the running time, QRAM, quantum memory, and classical memory. We optimise the QRW parameters for the first two inner calls, and also analyse the third inner call in detail. We compare our hybrid algorithm with the classical and pure quantum 4-sieves of Chailloux and Loyer at the same classical memory, and show that the hybrid achieves an exponential speedup over the classical 4-sieve in the low-memory regime.

1 Introduction

The Shortest Vector Problem (SVP) is at the foundation of lattice-based cryptography. The best known heuristic quantum algorithm for SVP runs in time $2^{0.2570d+o(d)}$ [1], obtained by replacing Grover’s algorithm inside the 2-sieve by a quantum random walk with an additional layer of locality-sensitive filtering. In parallel, k -sieve algorithms have been developed to reduce memory requirements at the cost of increased running time [2]. In particular, the quantum 4-sieve of [2] uses nested Grover searches and achieves a time-memory trade-off with quantum-accessible classical memory of size $2^{0.1724d}$ to $2^{0.1813d}$.

In this work, we investigate whether the QRW 2-sieve of [1] can be used as the inner two-list subroutine inside the 4-sieve framework of [2]. This requires adapting the QRW 2-sieve to handle two distinct lists rather than one, an arbitrary configuration threshold $C'_{12}(\alpha)$ rather than the fixed angle θ_α^* , and possibly non-unit vectors in the second-level call.

We show that the core QRW structure remains unchanged: the Johnson graph, the second random product code $\mathcal{C}_2$, the quantum data structure for the lists $J^v(\vec{t})$, and the MNRS framework are all preserved. Only the marked condition and the probability p_{sol} that a random cross-pair is a solution must be adjusted. We give the complete algorithms, discuss the modifications, and provide the full complexity analysis.

2 Preliminaries and Notation

We use the notation of [1, 2]. Let d be the lattice dimension and define the fundamental list-size parameter

$$N := 2^{cd+o(d)}. \quad (1)$$

This quantity N is exponential in d and serves as the reference scale for all exponents in the algorithm. A list of size N^a contains $2^{a \cdot cd+o(d)}$ elements; throughout the paper we write $N^{a+o(1)}$ to denote such a list, absorbing subexponential factors into the $o(1)$.

2.1 General notation

We write vectors in $\mathbb{R}^d$ as $\vec{x}$ and identify them with points. The Euclidean norm is denoted by $\|\cdot\|$. The unit sphere is

$$\mathcal{S}^{d-1} := \{\vec{x} \in \mathbb{R}^d : \|\vec{x}\| = 1\}. \quad (2)$$

For two vectors $\vec{x}_1, \vec{x}_2 \in \mathbb{R}^d$, their scalar product is $\langle \vec{x}_1 | \vec{x}_2 \rangle$ and the angle between them is

$$\theta(\vec{x}_1, \vec{x}_2) := \arccos\left(\frac{\langle \vec{x}_1 | \vec{x}_2 \rangle}{\|\vec{x}_1\| \|\vec{x}_2\|}\right). \quad (3)$$

We use the notation $\tilde{O}$ to suppress subexponential factors in d ; running times are expressed as $T = \tilde{O}(2^{cd})$.

For an angle $\alpha \in [0, \pi/2]$, the volume ratio of a spherical cap of angular radius α is

$$\mathcal{V}_d(\alpha) := \text{poly}(d) \sin^d(\alpha), \quad (4)$$

which represents the fraction of the unit sphere covered by a cap of angle α . For two angles α and θ , the volume ratio of a wedge formed by two caps of angle α with centers separated by angle θ is

$$\mathcal{W}_d(\alpha, \theta) := \text{poly}(d) \left(1 - \frac{2 \cos^2(\alpha)}{1 + \cos(\theta)}\right)^{d/2}, \quad (5)$$

which is well-defined provided $2 \cos^2(\alpha) < 1 + \cos(\theta)$, i.e., the two caps overlap.

The critical angle between residual vectors that characterises reducibility of two border points is

$$\theta_\alpha^* := 2 \arcsin\left(\frac{1}{2 \sin(\alpha)}\right), \quad (6)$$

as established in [1, Proposition 4]. Two residual vectors $\vec{y}_0, \vec{y}_1$ on the border of a filter of angle α are reducible if and only if $\theta(\vec{y}_0, \vec{y}_1) \leq \theta_\alpha^*$.

2.2 Quantum computing tools

QRAM model. The Quantum Random Access Memory (QRAM) is an operation added to the quantum circuit model. We consider quantum-accessible classical memory (QRACM). Given N classical registers $x_1, \dots, x_N \in \{0, 1\}^d$ stored in memory, a QRAM operation applies the unitary

$$U_{\text{QRAM}} : |i\rangle|y\rangle \rightarrow |i\rangle|x_i \oplus y\rangle. \quad (7)$$

We assume that for a given list L , there exists an efficient quantum circuit preparing

$$\frac{1}{\sqrt{|L|}} \sum_i |i\rangle|0\rangle \rightarrow \frac{1}{\sqrt{|L|}} \sum_i |i\rangle|L[i]\rangle. \quad (8)$$

This is achieved by applying Hadamard gates to $|0\rangle$ to create a uniform superposition over indices, followed by querying $L[i]$ for each index in superposition.

Grover's algorithm. Given QRAM access to a list $L = \{x_1, \dots, x_\ell\}$ and a function $f : L \rightarrow \{0, 1\}$ with t marked elements satisfying $f(x_i) = 1$, Grover's algorithm returns a marked element with probability greater than $1/2$ using $\mathcal{O}(\sqrt{|L|/t})$ calls to the unitary $O_f : |x\rangle|y\rangle \rightarrow |x\rangle|y \oplus f(x)\rangle$. Classically, no algorithm can solve this with fewer than $\Theta(|L|/t)$ queries in the black-box model.

Quantum amplitude amplification. Let $\mathcal{A}$ be an algorithm without measurements that finds a solution $x \in L$ with $f(x) = 1$ with success probability p . Quantum amplitude amplification returns a solution with probability $1/2$ using $\mathcal{O}(1/\sqrt{p})$ calls to O_f .

Both Grover's algorithm and amplitude amplification can be made to succeed with probability $1 - 2^{-\eta}$ by repeating them $\mathcal{O}(\eta)$ times.

2.3 Lattice sieving

Lattices and SVP. Given a basis $B = (\vec{b}_1, \dots, \vec{b}_m) \in \mathbb{R}^d$ of linearly independent vectors, the lattice generated by B is

$$\mathcal{L}(B) := \left\{ \sum_{i=1}^m z_i \vec{b}_i : z_i \in \mathbb{Z} \right\}. \quad (9)$$

For simplicity we work with full-rank lattices, i.e., $d = m$. The Shortest Vector Problem (SVP) asks to find a shortest non-zero vector of $\mathcal{L}(B)$. By Minkowski's theorem, the Euclidean norm of a shortest vector is upper-bounded by $\sqrt{d} \cdot \det(B)^{1/d}$.

Sieving algorithms. Sieving algorithms, introduced by Nguyen and Vidick, solve SVP heuristically. Given a list of lattice vectors of norm at most R and a reducing factor $\gamma < 1$, a sieving step returns a list of lattice vectors of norm at most γR . To obtain these reduced vectors, one computes all differences between pairs of vectors of the input list and keeps those of norm at most γR . Iterating this step builds lists of progressively shorter lattice vectors. The initial list can be sampled using Klein's algorithm. Because the norms reduce by a factor γ at each step, the output list will contain a non-zero shortest lattice vector after a polynomial number of iterations.

We normalise $R = 1$ and $\gamma \approx 1$ (typically $\gamma = 1 - 1/\text{poly}(d)$) for simplicity; these choices do not affect the asymptotic analysis. We note that Gauss-sieve algorithms have worse asymptotic time-memory trade-offs and are not analysed here, although they are often faster in practice. Our analysis relies on the following heuristic.

Remark 1 (Heuristic). *Lattice points behave like uniform points on the sphere $\mathcal{S}^{d-1}$.*

This heuristic has been studied by Nguyen and Vidick. It becomes invalid when the vectors become short, but in that case we can assume SVP is solved.

Tuple-sieve and the k -Lists problem. The NV-sieve generalises to the k -sieve. The approximate k -Lists problem is defined as follows: given k lists $L_1, \dots, L_k$ of equal exponential size N whose elements are i.i.d. uniform vectors from $\mathcal{S}^{d-1}$, find N k -tuples $(\vec{x}_1, \dots, \vec{x}_k) \in L_1 \times \dots \times L_k$ satisfying

$$\|\vec{x}_1 + \dots + \vec{x}_k\| \leq 1. \quad (10)$$

The spherical cap of center $\vec{s}$ and angle α is

$$\mathcal{H}_{\vec{s}, \alpha} := \{\vec{x} \in \mathcal{S}^{d-1} : \theta(\vec{x}, \vec{s}) \leq \alpha\}. \quad (11)$$

For an angle $\alpha \in [0, \pi/2]$ and $\vec{v} \in \mathcal{S}^{d-1}$, the ratio of the volume of $\mathcal{H}_{\vec{v}, \alpha}$ to the volume of $\mathcal{S}^{d-1}$ is $\mathcal{V}_d(\alpha) = \text{poly}(d) \sin^d(\alpha)$.

2.4 Configurations

In the approximate k -Lists problem, the condition $\|\vec{x}_1 + \dots + \vec{x}_k\| \leq 1$ can be rewritten as

$$\|\vec{x}_1 + \dots + \vec{x}_k\|^2 = \sum_{i=1}^k \|\vec{x}_i\|^2 + 2 \sum_{i,j \neq i} \langle \vec{x}_i | \vec{x}_j \rangle. \quad (12)$$

This means the norm condition can be verified through constraints on the pairwise scalar products $\langle \vec{x}_i | \vec{x}_j \rangle$. This motivates the following definition.

Definition 2 (Configuration). *The configuration C of k points $\vec{x}_1, \dots, \vec{x}_k \in \mathcal{S}^{d-1}$ is the Gram matrix of the $\vec{x}_i$'s, i.e., $C_{i,j} = \langle \vec{x}_i | \vec{x}_j \rangle$. A configuration is said to be balanced when $C_{i,j} = -1/k$ for $i \neq j$ and $C_{i,i} = 1$. In this case, the tuple of points forms the vertices of a regular polyhedron inscribed in the sphere. For $I \subset [k]$, we denote by $C[I]$ the $|I| \times |I|$ submatrix of C obtained by restricting to rows and columns indexed by I .*

Definition 3 (Configuration problem). *Let $k \in \mathbb{N}$ and $\epsilon > 0$. Suppose we are given a target configuration $C \in \mathbb{R}^{k \times k}$. Given lists $L_1, \dots, L_k$ all of exponential size $|L|$ whose elements are i.i.d. uniform from $\mathcal{S}^{d-1}$, the configuration problem consists of finding a $1 - o(1)$ fraction of all solutions, where a solution is a k -tuple $(\vec{x}_1, \dots, \vec{x}_k)$ with $\vec{x}_i \in L_i$ such that $\langle \vec{x}_i | \vec{x}_j \rangle \leq C_{i,j}$ for all i, j .*

Herold and Kirshanova showed that the approximate k -Lists problem reduces to the configuration problem. The probability that a k -tuple of i.i.d. uniform points on $\mathcal{S}^{d-1}$ satisfies a given configuration $C \in \mathbb{R}^{k \times k}$ is

$$\det(C)^{d/2}. \quad (13)$$

After a sieving step on a list L , we want to obtain $|L|^k \cdot \det(C)^{d/2}$ k -tuples satisfying the chosen configuration C , so that they are reducing k -tuples. We need this number to equal $|L|$, which gives the required size of L in terms of the target configuration:

$$|L| = \tilde{\mathcal{O}}\left(\left(\frac{1}{\det(C)}\right)^{\frac{d}{2(k-1)}}\right). \quad (14)$$

For the analysis of filtered lists, we need the expected size of the sublist of L_j satisfying constraints with respect to previously chosen vectors. Given a configuration $C \in \mathbb{R}^{k \times k}$ and lists $L_1, \dots, L_k \subset \mathcal{S}^{d-1}$ of sizes $|L_j|$, for $\vec{x}_1, \dots, \vec{x}_i \in \mathcal{S}^{d-1}$ we denote

$$L_j(\vec{x}_1, \dots, \vec{x}_i) := \{\vec{x}_j \in L_j : \langle \vec{x}_1 | \vec{x}_j \rangle \leq C_{1,j}, \dots, \langle \vec{x}_i | \vec{x}_j \rangle \leq C_{i,j}\}. \quad (15)$$

Then, for an i -tuple $\vec{x}_1, \dots, \vec{x}_i$ satisfying the configuration $C[1 \dots i]$, the expected size of $L_j(\vec{x}_1, \dots, \vec{x}_i)$ is

$$\mathbb{E}(|L_j(\vec{x}_1, \dots, \vec{x}_i)|) = |L_j| \cdot \left(\frac{\det(C[1, \dots, i, j])}{\det(C[1, \dots, i])}\right)^{d/2}. \quad (16)$$

In particular,

$$\mathbb{E}(|L_j(\vec{x}_i)|) = |L_j| \cdot (1 - C_{i,j}^2)^{d/2}. \quad (17)$$

2.5 The k -sieve framework

The general framework for k -sieve algorithms, introduced in [2], proceeds in three steps: prefilter the list vectors, search all reduced tuples within each filter, and repeat until all reduced points are found. The algorithm takes as input an angle α and a configuration $C \in \mathbb{R}^{k \times k}$ that defines constraints over the lattice vectors. From α and C , one computes the configuration $C'(\alpha)$ over the residual vectors on filters of angle α using the transformation

$$C'_{i,j}(\alpha) = -\frac{1}{\sin^2(\alpha)} \left(C_{i,j} + \frac{\cos^2(\alpha)}{k-1} \right) \quad (i \neq j). \quad (18)$$

From C , the minimum memory requirement is $|L| = \tilde{\mathcal{O}}(\det(C)^{-\frac{1}{2(k-1)}})$, as in Equation (14).

Prefiltering. Sample a k -RPC $\mathfrak{C}$ of size $k \cdot 1/\mathcal{V}_d(\alpha)$. Its codewords are $(\mathbf{A}_i^j)_{i,j}$ for $i \in [k]$ and $j \in [| \mathfrak{C} | / k]$, with $\langle \mathbf{A}_{i_1}^j | \mathbf{A}_{i_2}^j \rangle = -1/(k-1)$ for $i_1 \neq i_2$, which implies $\sum_{i=1}^k \mathbf{A}_i^j = \vec{0}$. For each vector $\vec{x} \in L$, compute its nearest codeword in $\mathfrak{C}$. If the nearest codeword is $\mathbf{A}_i^j$, add $\vec{x}$ to the associated list L_i^j , and add the residual vector $\vec{y} = (\vec{x} - \cos(\alpha)\mathbf{A}_i^j)/\sin(\alpha)$ to the list R_i^j . The number of tuple-filters is

$$\text{NbFilters} := |\mathfrak{C}|/k = \mathcal{O}\left(\frac{1}{\mathcal{V}_d(\alpha)}\right). \quad (19)$$

The prefiltering step takes time $|L|$ in total.

Finding all solutions within a tuple-filter. For a fixed $j \in [\mathfrak{C}/k]$, consider the configuration problem on the k lists $(R_i^j)_i$ with configuration $C'(\alpha)$. The subroutine must find all k -tuples within $R_1^j \times \dots \times R_k^j$ satisfying $C'(\alpha)$. The expected number of such tuples is given by the following lemma.

Lemma 4 (Expected solutions per filter). *With the notations above and for fixed $j \in [\mathfrak{C}/k]$, the expected number of tuples in the tuple-filter associated with the lists $R_1 \times \dots \times R_k$ satisfying configuration $C'(\alpha)$ is on average*

$$|\text{Sol}_f| = \mathcal{O}\left(|R_1|^k \cdot \det(C'(\alpha))^{d/2}\right) = \mathcal{O}\left(|L|^k \mathcal{V}_d(\alpha)^k \cdot \det(C'(\alpha))^{d/2}\right). \quad (20)$$

Proof. There are $|R_1|^k$ tuples in $R_1 \times \dots \times R_k$ since the lists all have the same size $|R_1| = |L| \cdot \mathcal{V}_d(\alpha)$. Any tuple $(\vec{y}_1, \dots, \vec{y}_k)$ from this set has probability $\det(C'(\alpha))^{d/2}$ to satisfy configuration $C'(\alpha)$. Hence the expected number of satisfying tuples is as claimed. $\square$

Any subroutine with these inputs and outputs may suit the framework. We denote the time complexity of the subroutine that finds all solutions within a filter with parameters α and C by $T(\mathbf{FAS}_{C'(\alpha)})$.

Number of repetitions. After searching all solutions within every tuple-filter, the expected total number of solutions is

$$|\text{Sol}_{\text{all}}| = |L|^k \cdot \det(C)^{d/2}. \quad (21)$$

To complete the sieve step, we require $|L|$ reduced lattice vectors. Thus steps 1 and 2 must be repeated until enough solutions have been found. The missed solutions are those for which part of the solution lies in one tuple-filter and the rest in another. By performing a new prefiltering, the partition of the sphere changes, allowing some of these missing solutions to be found.

Lemma 5 (Number of repetitions). *The number of repetitions in the outer loop is*

$$\text{NbRep}_{\alpha, C} = \mathcal{O}\left(\max\left\{1, \frac{|\text{Sol}_{\text{all}}|}{|\text{Sol}_f|} \cdot \text{NbFilters}\right\}\right) = \mathcal{O}\left(\max\left\{1, \frac{\det(C)^{d/2}}{\mathcal{V}_d^{k-1}(\alpha) \det(C'(\alpha))^{d/2}}\right\}\right). \quad (22)$$

Proof. From Equations (21), (19), and Lemma 4, the ratio $|\text{Sol}_{\text{all}}|/(|\text{Sol}_f| \cdot \text{NbFilters}^{-1})$ equals

$$\frac{|L|^k \det(C)^{d/2}}{|L|^k \mathcal{V}_d^k(\alpha) \det(C'(\alpha))^{d/2} \cdot \mathcal{V}_d(\alpha)} = \frac{\det(C)^{d/2}}{\mathcal{V}_d^{k-1}(\alpha) \det(C'(\alpha))^{d/2}}, \quad (23)$$

which gives the claimed bound. $\square$

The overall time complexity of an algorithm based on this framework is given by the following theorem.

Theorem 6 (Framework complexity). *Let $\alpha \in (0, \pi/2]$ be an angle and $C \in \mathbb{R}^{k \times k}$ a configuration, and let $C'(\alpha)$ be the configuration on the residual vectors given by Equation (18). Given an algorithm that solves the configuration problem $C'(\alpha)$ for k lists in time $T(\mathbf{FAS}_{C'(\alpha)})$, the sieve algorithm solves SVP in time*

$$T(k\text{-sieve}) := \text{NbRep}_{\alpha, C} \cdot (|L| + \text{NbFilters}_{\alpha} \cdot T(\mathbf{FAS}_{C'(\alpha)})), \quad (24)$$

where $\text{NbRep}_{\alpha, C}$ is given by Lemma 5 and $\text{NbFilters}_{\alpha} = \mathcal{O}(1/\mathcal{V}_d(\alpha))$ by Equation (19).

The main novelty of this framework is the angle α , which can be freely chosen. Taking $\alpha = \pi/2$ means that no tailored LSF is performed. The configuration entries $C_{i,j}$ are parameters to be optimised to minimise the overall time of the k -sieve, subject to the constraints on memory and reducibility of the tuples. We also require that the inner algorithm for solving the configuration problem with $C'(\alpha)$ uses at most memory M . The prefiltering angle $\alpha \in (0, \pi/2]$ is optimised numerically.

2.6 Outer 4-sieve parameters

The outer 4-sieve uses a prefiltering angle $\alpha_0 \in [\pi/3, \pi/2]$ and a configuration matrix $C \in \mathbb{R}^{4 \times 4}$. The reduction to residual vectors is denoted $C'(\alpha_0)$, with entries $C'_{ij}(\alpha_0)$ given by Equation (18):

$$C'_{ij}(\alpha_0) = \frac{1}{\sin^2(\alpha_0)} \left(C_{ij} + \frac{\cos^2(\alpha_0)}{3} \right) \quad (i \neq j). \quad (25)$$

The effective threshold for the second-level 2-sieve that builds L_{1234} is

$$C_{\text{eff}} := \frac{\sin^2(\alpha_0)}{2R^2} - 1, \quad R^2 := 2 + 2C_{12}, \quad (26)$$

as given in [2, Lemma 6].

2.7 QRW parameters

The adapted QRW 2-sieve uses four free parameters c, c_1, c_2, ρ satisfying $0 \leq c_2 \leq c_1 \leq c \leq 1$ and $0 \leq \rho \leq \rho_0$. From these, the following derived quantities are defined. The angle α (not to be confused with α_0) satisfies $\mathcal{V}_d(\alpha) = N^{-(1-c)}$. The angle β of the second random product code satisfies $\mathcal{V}_d(\beta) = N^{c_2-c_1}$. The target angle is $\theta' := \arccos(C')$. The collision parameter ρ_0 is defined by $N^{\rho_0} := \mathcal{V}_d(\beta)/\mathcal{W}_d(\beta, \theta')$. The solution probability is

$$p_{\text{sol}} := \frac{1}{2} (1 - (C')^2)^{d/2}, \quad (27)$$

where the factor $1/2$ accounts for the bipartite nature of the problem. The expected number of solutions per filter is $N^\zeta := N^{2c} \cdot p_{\text{sol}}$.

The QRW costs are the setup cost $\mathcal{S} := N^{c_1+\rho}$, the update cost $\mathcal{U} := N^{\max\{\rho, (\rho+c_2)/2\}}$, and the checking cost $\mathcal{C} := 1$. The spectral gap of the Johnson graph is $\delta := N^{-c_1}$. The fraction of marked vertices is

$$\epsilon := N^{2c_1} \cdot p_{\text{sol}} \cdot N^{\rho-\rho_0}. \quad (28)$$

3 Modified QRW 2-Sieve

3.1 Preserved components

The adaptation preserves the following components of the QRW 2-sieve of [1] without structural modification. The Johnson graph $J(N, N^{c_1})$ on the combined list, with spectral gap $\delta = N^{-c_1}$ and the MNRS quantum walk framework, is retained exactly. The second random product code $\mathcal{C}_2$ with angle β satisfying $\mathcal{V}_d(\beta) = N^{c_2-c_1}$ and size $|\mathcal{C}_2| = N^{\rho+c_1-c_2}$ is constructed identically. The quantum data structure for the lists $J^v(\vec{t}) = f_\beta(\vec{t}) \cap L^v$, supporting insertions and deletions in quantum superposition, is used as in the original algorithm. The setup, update, and check cost formulas are formally unchanged.

3.2 Required modifications

Three modifications are required to adapt the QRW 2-sieve to the bipartite setting. First, we form the combined list $L = R_A \cup R_B$ and tag each element by a bit indicating its origin. The Johnson graph is built on L with $|L| = 2N_R$. This tagging does not affect the graph structure. Second, the marked condition is redefined: a vertex v is marked if there exist $\vec{t} \in \mathcal{C}_2$ and distinct $\vec{y}_i, \vec{y}_j \in \tilde{J}^v(\vec{t})$ such that $\vec{y}_i$ and $\vec{y}_j$ have different tags and $\langle \vec{y}_i | \vec{y}_j \rangle \leq C'$. Third, the solution probability acquires the factor $1/2$ as given in Equation (27).

3.3 Adapted algorithm

Algorithm 1 Adapted QRW 2-Sieve for bipartite configuration problems

Require: two residual lists R_A, R_B of size N_R each; threshold C' ; QRW parameters

c, c_1, c_2, ρ .

Ensure: all pairs $(\vec{y}_a, \vec{y}_b) \in R_A \times R_B$ with $\langle \vec{y}_a | \vec{y}_b \rangle \leq C'$.

- 1: Form $L := R_A \cup R_B$ with tags.
 - 2: Compute $\theta' := \arccos(C')$ and $p_{\text{sol}} := \frac{1}{2}(1 - C'^2)^{d/2}$.
 - 3: Sample random product code $\mathcal{C}_2$ with $|\mathcal{C}_2| = N^{\rho+c_1-c_2}$ and angle β satisfying $\mathcal{V}_d(\beta) = N^{c_2-c_1}$.
 - 4: Build Johnson graph $J(N, N^{c_1})$ on L .
 - 5: **for** each vertex v **do**
 - 6: Compute $J^v(\vec{t}) = f_\beta(\vec{t}) \cap L^v$ for each $\vec{t} \in \mathcal{C}_2$.
 - 7: Truncate to $\tilde{J}^v(\vec{t})$ of size $\leq 2N^{c_2}$.
 - 8: Set marked bit $M(v)$ according to the marked condition above.
 - 9: **end for**
 - 10: **while** total solutions found $< N^\zeta/2$ **do**
 - 11: Pick a new random product code $\mathcal{C}_2$.
 - 12: **while** solutions found with this $\mathcal{C}_2 < N^{\zeta+\rho-\rho_0}/2$ **do**
 - 13: Run the MNRS quantum walk to find one marked vertex.
 - 14: **end while**
 - 15: **end while**
 - 16: **return** all collected pairs.
-

4 Hybrid Quantum 4-sieve

Algorithm 2 Hybrid Quantum 4-sieve

Require: list L of $N' = N^{1+o(1)}$ lattice vectors of norm 1; reducing factor $\gamma < 1$; prefiltering angle α_0 ; configuration $C \in \mathbb{R}^{4 \times 4}$; QRW parameters.

Ensure: list L' of N' lattice vectors of norm $\leq \gamma$.

- 1: $L' := \emptyset$.
 - 2: **while** $|L'| \leq N'$ **do**
 - 3: Sample a 4-RPC $\mathfrak{C}$ with codewords $(\mathbf{A}_1^j, \mathbf{A}_2^j, \mathbf{A}_3^j, \mathbf{A}_4^j)$ such that $\sum_i \mathbf{A}_i^j = \vec{0}$ and $\langle \mathbf{A}_i^j | \mathbf{A}_{i'}^j \rangle = -1/3$ for $i \neq i'$.
 - 4: Partition L into tuple-filters: for each $\vec{x} \in L$, find its nearest codeword $\mathbf{A}_i^j$, add $\vec{x}$ to L_i^j , compute residual $\vec{y} = (\vec{x} - \cos \alpha_0 \mathbf{A}_i^j) / \sin \alpha_0$, and add $\vec{y}$ to R_i^j .
 - 5: **for** each tuple-filter j **do**
 - 6: $L_{12}^j \leftarrow \text{QRW-2-SIEVE}(R_1^j, R_2^j, C'_{12}(\alpha_0))$.
 - 7: $L_{34}^j \leftarrow \text{QRW-2-SIEVE}(R_3^j, R_4^j, C'_{34}(\alpha_0))$.
 - 8: $L_{1234}^j \leftarrow \text{GROVER}(\tilde{L}_{12}^j, \tilde{L}_{34}^j, C_{\text{eff}})$.
 - 9: $L' := L' \cup L_{1234}^j$.
 - 10: **end for**
 - 11: **end while**
 - 12: **return** L' .
-

5 Complexity Analysis

We now give the full complexity of the adapted QRW 2-sieve and of the outer 4-sieve. All calculations are performed in the QRAM model and expressed in the exponent notation $N^{cd+o(1)}$.

5.1 Complexity of the adapted QRW 2-sieve

The setup cost, update cost, checking cost, and spectral gap are formally unchanged from the original algorithm:

$$\mathcal{S} = N^{c_1 + \rho + o(1)}, \quad (29)$$

$$\mathcal{U} = N^{\max\{\rho, (\rho + c_2)/2\} + o(1)}, \quad (30)$$

$$\mathcal{C} = 1, \quad (31)$$

$$\delta = N^{-c_1}. \quad (32)$$

The fraction of marked vertices is modified by the factor $1/2$ in p_{sol} :

$$\epsilon = \frac{1}{2} N^{2c_1} (1 - (C')^2)^{d/2} N^{\rho - \rho_0}, \quad (33)$$

where ρ_0 satisfies $N^{\rho_0} = \mathcal{V}_d(\beta)/\mathcal{W}_d(\beta, \theta')$. The expected number of solutions per filter is

$$N^\zeta = \frac{1}{2} N^{2c} (1 - (C')^2)^{d/2}. \quad (34)$$

The time to find one marked vertex is

$$T_1 = \mathcal{S} + \frac{1}{\sqrt{\epsilon}} \left(\frac{1}{\sqrt{\delta}} \mathcal{U} + \mathcal{C} \right). \quad (35)$$

The time for the FINDALLSOLUTIONS subroutine is

$$\text{FAS}_1 = \max\{N^\zeta, 1\} \cdot T_1. \quad (36)$$

The total time of one QRW 2-sieve call is

$$T_{\text{QRW}} = \text{NbRep}_{\text{QRW}} \cdot (N + N^{1-c} \cdot \text{FAS}_1), \quad (37)$$

where

$$\text{NbRep}_{\text{QRW}} = \max\{1, N^{c-\zeta+o(1)}\}. \quad (38)$$

The derived parameters are:

$$\log_2 \sin(\alpha) = -(1 - c) \cdot n, \quad (39)$$

$$\log_2 \sin(\beta) = (c_2 - c_1) \cdot n, \quad (40)$$

$$\theta' = \arccos(C'), \quad (41)$$

$$\rho_0 = \frac{\log_2 \mathcal{V}_d(\beta) - \log_2 \mathcal{W}_d(\beta, \theta')}{n d}, \quad (42)$$

$$\zeta = 2c + \frac{\log_2 p_{\text{sol}}}{n d}. \quad (43)$$

5.2 QRW 2-sieve on the standard 2-sieve problem

We first verify the framework by reproducing the result of [1, Theorem 1] for the standard 2-sieve problem. In this case, the list size is N (i.e., $n = 0.2075$), the threshold is θ_α^* with $p_{\text{sol}} = \mathcal{V}_d(\theta_\alpha^*)$, and the optimal parameters are

$$c \approx 0.3696, \quad c_1 \approx 0.2384, \quad c_2 = 0, \quad \rho \rightarrow 0. \quad (44)$$

With these values:

$$\log_2 \sin(\alpha) = -0.1308, \quad \alpha \approx 1.1514 \text{ rad}, \quad (45)$$

$$\log_2 \sin(\beta) = -0.0495, \quad \beta \approx 1.313 \text{ rad}, \quad (46)$$

$$\theta_\alpha^* \approx 1.158 \text{ rad}, \quad \rho_0 \approx 0.106, \quad \zeta \approx 0.132. \quad (47)$$

The QRW costs are:

$$\mathcal{S} = N^{0.2384}, \quad \mathcal{U} = N^{o(1)}, \quad \delta = N^{-0.2384}, \quad \epsilon = N^{-0.2364}. \quad (48)$$

Then $T_1 \approx N^{0.2384}$, $\text{FAS}_1 = N^{0.3704}$, and

$$T_{\text{QRW}} \approx N^{1.2384} = 2^{0.2570d + o(d)}, \quad (49)$$

which matches [1, Theorem 1].

5.3 QRW 2-sieve for the first two inner calls

We now apply the framework to the first two inner calls of the 4-sieve at the minimal-memory configuration of [2, Proposition 17]: $|L| = 2^{0.1724d}$, $\alpha_0 = 1.350196$ rad, and balanced configuration $C_{ij} = -1/4$ for $i \neq j$. The residual list size is $|R_i| = 2^{0.137008d}$. The threshold is

$$C'_{12}(\alpha_0) = \frac{-1/4 + \cos^2(1.350196)/3}{\sin^2(1.350196)} = -0.245809. \quad (50)$$

The solution probability is

$$p_{12} = (1 - C'_{12})^{d/2} = 2^{-0.044958d}. \quad (51)$$

For the combined list $N_1 = 2|R_i|$ with $n_1 = 0.137008$, we have $\log_{N_1} p_{12} \approx -0.32815$. We optimise the QRW parameters. The optimal choice is

$$c = 0.3083, \quad c_1 = 0.1000, \quad c_2 = 0.0000, \quad \rho = 0. \quad (52)$$

With these values, we compute the collision parameter:

$$\log_2 \sin(\beta) = (c_2 - c_1) \cdot n_1 = -0.013701, \quad (53)$$

$$\cos^2(\beta) \approx 0.02027, \quad (54)$$

$$1 + \cos(\theta') = 1 - 0.245809 = 0.754191, \quad (55)$$

$$\log_2 \mathcal{W}_d(\beta, \theta')/d \approx -0.03737, \quad (56)$$

$$\rho_0 = \frac{-0.013701 - (-0.03737)}{0.137008} = 0.169462. \quad (57)$$

The marked fraction is

$$\epsilon = N_1^{2c_1} \cdot p_{12} \cdot N_1^{\rho - \rho_0} = N_1^{0.2 - 0.32815 - 0.169462} = N_1^{-0.297612}. \quad (58)$$

The remaining quantities are

$$\zeta = 2c + \log_{N_1} p_{12} = 0.6166 - 0.32815 = 0.288526, \quad (59)$$

$$T_1 = N_1^{0.1} + N_1^{0.148806} (N_1^0 + 1) \approx N_1^{0.198801}, \quad (60)$$

$$\text{FAS}_1 = N_1^{0.288526} \cdot N_1^{0.198801} = N_1^{0.487327}, \quad (61)$$

$$\text{NbRep}_{\text{QRW}} = N_1^{0.3083 - 0.288526} = N_1^{0.019807}, \quad (62)$$

$$T_{\text{QRW}} = N_1^{0.019807} (N_1 + N_1^{0.6917} \cdot N_1^{0.487327}) \approx N_1^{1.198801}. \quad (63)$$

Converting $T_{12} = T_{34} = 2^{0.137008 \times 1.198801 d + o(d)} = 2^{0.164245d + o(d)}$.

5.4 Complexity of the Third Inner Call

After the first two inner calls, we have obtained the lists L_{12} and L_{34} of pairs of residual vectors that satisfy the firstlevel configuration constraints $C'_{12}(\alpha_0)$ and $C'_{34}(\alpha_0)$, respectively. The third inner call consists in finding all pairs

$$(\mathbf{z}_{12}, \mathbf{z}_{34}) \in L_{12} \times L_{34}$$

such that

$$\langle \mathbf{z}_{12} | \mathbf{z}_{34} \rangle \leq C_{\text{eff}}, \quad C_{\text{eff}} := \frac{\sin^2(\alpha_0)}{2R^2} - 1, \quad R^2 := 2 + 2C_{12}, \quad (64)$$

where α_0 is the outer prefiltering angle and C_{12} is the outer configuration entry. This condition is exactly the one given in [2, Lemma 6] and guarantees that the four residual vectors form a reducing tuple.

Probability of a solution. For two random vectors $\mathbf{z}_{12}$ and $\mathbf{z}_{34}$ of norm R , the probability that their inner product is at most C_{eff} is

$$p_{\text{sol}} = (1 - C_{\text{eff}}^2)^{d/2}. \quad (65)$$

There is no factor $1/2$ here, because the two lists L_{12} and L_{34} are distinct and the search is over ordered pairs $(\mathbf{z}_{12}, \mathbf{z}_{34})$.

The expected number of solutions per tuplefilter is therefore

$$|\text{Sol}| = |L_{12}|^2 p_{\text{sol}} = |L_{12}|^2 (1 - C_{\text{eff}}^2)^{d/2}. \quad (66)$$

Grover search. We use Grover's algorithm to find all solutions in $L_{12} \times L_{34}$. The search space has size $|L_{12}|^2$. The number of Grover iterations to find one solution is $O(1/\sqrt{p_{\text{sol}}})$, and to find all $|\text{Sol}|$ solutions we use amplitude amplification, which requires

$$T_{1234} = |L_{12}|^2 (1 - C_{\text{eff}}^2)^{d/4} = |L_{12}|^2 \sqrt{p_{\text{sol}}} \quad (67)$$

elementary operations.

Numerical values for the minimal-memory configuration. We take the minimal-memory parameters of [2, Proposition 17]:

$$|L| = 2^{0.1724d}, \quad \alpha_0 = 1.350196 \text{ rad}, \quad C_{ij} = -\frac{1}{4} \ (i \neq j).$$

Then

$$\sin^2(\alpha_0) = 0.952120, \quad R^2 = 1.5, \quad C_{\text{eff}} = -0.682627.$$

The first-level threshold is

$$C'_{12}(\alpha_0) = \frac{-\frac{1}{4} + \frac{\cos^2(\alpha_0)}{3}}{\sin^2(\alpha_0)} = -0.245809.$$

The residual list size is

$$|R_i| = 2^{0.137008d},$$

so that

$$|L_{12}| = |R_i|^2 (1 - C'_{12}(\alpha_0)^2)^{d/2} = 2^{0.229057d}.$$

The solution probability is

$$p_{\text{sol}} = (1 - C_{\text{eff}}^2)^{d/2} = 2^{-0.452516d},$$

hence

$$T_{1234} = 2^{0.229057d \cdot 2} \cdot 2^{-0.452516d/2} = 2^{0.231856d + o(d)}.$$

The expected number of solutions per tuple-filter is

$$|\text{Sol}| = |L_{12}|^2 p_{\text{sol}} = 2^{0.005598d}.$$

From Section 5.3, the first two inner calls cost

$$T_{12} = T_{34} \approx 2^{0.164245d + o(d)}.$$

Thus the total time per tuple-filter is dominated by the third call:

$$T_{\text{filter}} = T_{12} + T_{34} + T_{1234} \approx 2^{0.231856d + o(d)}. \quad (68)$$

Finally, we multiply by the number of tuple-filters and the number of repetitions of the outer loop. For the minimal-memory configuration with $\alpha_0 = 1.350196$ rad, we have

$$\text{NbFilters} = 2^{0.0354d}, \quad \text{NbRep} = 2^{0.0780d}.$$

Table 1: Comparison of 4-sieve algorithms at different classical memory constraints.

Classical memory	Classical 4-sieve [2]	Hybrid quantum 4-sieve (ours)	quantum 4-sieve [2]
$2^{0.1724d}$	$2^{0.418d}$	$2^{0.345d}$	$2^{0.3276d}$
$2^{0.1813d}$	$2^{0.???d}$	$2^{0.???d}$	$2^{0.3120d}$
$2^{0.1887d}$	$2^{0.324d}$	$2^{0.???d}$	$2^{0.???d}$
$2^{0.2075d}$	$2^{0.2925d}$	$2^{0.???d}$	$2^{0.???d}$

Therefore the total running time of the hybrid quantum 4-sieve is

$$T_{\text{hybrid}} = \text{NbRep} \cdot \text{NbFilters} \cdot T_{\text{filter}} = 2^{0.0780d} \cdot 2^{0.0354d} \cdot 2^{0.231856d} = 2^{0.345301d+o(d)}. \quad (69)$$

We round this to

$$T_{\text{hybrid}} \approx 2^{0.345d+o(d)}.$$

At the minimal classical memory $M = 2^{0.1724d}$, the classical 4-sieve runs in time $2^{0.418d+o(d)}$, whereas our hybrid algorithm runs in time $2^{0.345d+o(d)}$. The improvement factor is

$$2^{0.418d-0.345d} = 2^{0.073d},$$

which is an exponential speedup. For comparison, the pure quantum 4-sieve of [2] achieves $2^{0.3276d+o(d)}$ at the same memory, so our hybrid is slower by $2^{0.0174d}$ than the best known quantum 4-sieve.

The advantage of the hybrid is most pronounced in the low-memory regime. As the allowed memory increases, the classical 4-sieve improves rapidly: at $M = 2^{0.1887d}$ it runs in $2^{0.324d}$, and at $M = 2^{0.2075d}$ it reaches the classical 2-sieve time $2^{0.2925d}$. Our hybrid, optimized for the minimal-memory point, does not immediately benefit from additional memory without a re-optimization of the QRW parameters. A full time-memory trade-off curve for the hybrid would require running the parameter optimization for several values of α_0 and C , which we leave for future work.

In summary, the hybrid quantum 4-sieve demonstrates that quantum random walk techniques can be successfully integrated into the k -sieve framework. While it does not surpass the best pure quantum 4-sieve, it offers a significant quantum advantage over the classical 4-sieve in the regime where memory is severely constrained.

References

- [1] A. Chailloux and J. Loyer. Lattice sieving via quantum random walks. In *ASIACRYPT 2021*, 2021.
- [2] A. Chailloux and J. Loyer. Classical and quantum 3 and 4-sieves to solve SVP with low memory. In *PQCrypto 2023*, pp. 225–255, 2023.